

Graphical Galois Slicing

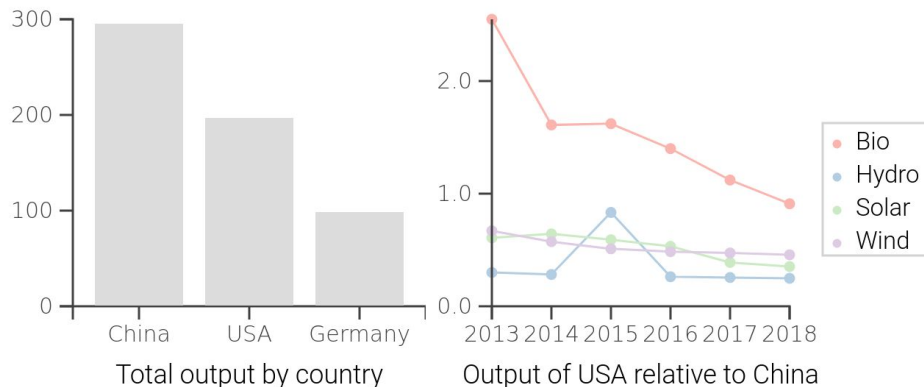
Joseph Bond, Cristina David, Minh Nguyen and Roly Perera

Contents

1. Background and Problem Statement
2. Work to date
3. Work in Progress
4. Conclusion and Future Work

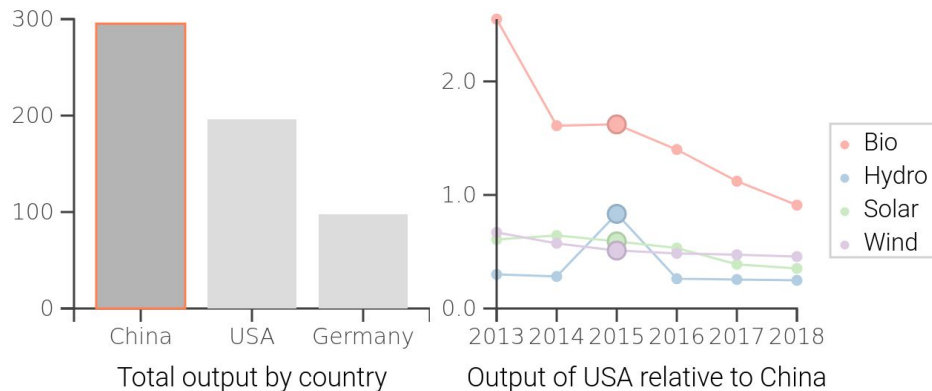
Background: Fluid

- Data analytics involves extensive use of visualizations
 - Understanding visualizations is a subtle business
 - Goal is to understand where selected visual elements come from
- Fluid is a language which bakes in this sort of analysis as standard ⁽¹⁾



Background: Fluid

- Data analytics involves extensive use of visualizations
 - Understanding visualizations is a subtle business
 - Goal is to understand where selected visual elements come from
- Fluid is a language which bakes in this sort of analysis as standard ⁽¹⁾



common data				
#	year	country	energyType	output
25	2015	China	Bio	10.3
26	2015	China	Hydro	96
27	2015	China	Solar	44
28	2015	China	Wind	145

Bidirectional dependency analysis

image

15	13	6	9	16
12	5	15	4	13
14	9	20	8	1
4	10	3	7	19
3	11	15	2	9

filter

-2	-1	0
-1	1	1
0	1	2

bwd = demand

```

1 let emboss = [[-2, -1, 0],
2               [-1, 1, 1],
3               [ 0, 1, 2]];
4   filter = [| nth2 i j emboss
5             | (i, j) in (3, 3) |];
6   image' = [[15, 13, 6, 9, 16],
7             [12, 5, 15, 4, 13],
8             [14, 9, 20, 8, 1],
9             [ 4, 10, 3, 7, 19],
10            [ 3, 11, 15, 2, 9]];
11   image = [| nth2 i j image'
12            | (i, j) in (5, 5) |]
13 in convolve image filter zero
    
```

output

5	4	2	5	2
3	1	2	-1	-2
3	0	1	0	-1
2	1	-2	0	0
1	0	-1	-1	-2

image

15	13	6	9	16
12	5	15	4	13
14	9	20	8	1
4	10	3	7	19
3	11	15	2	9

filter

-2	-1	0
-1	1	1
0	1	2

fwd = sufficiency

output

5	4	2	5	2
3	1	2	-1	-2
3	0	1	0	-1
2	1	-2	0	0
1	0	-1	-1	-2

How does Fluid Work?

- Apply *dynamic program slicing* techniques to inputs and outputs
 - Backward slice against selection represents *necessity*
 - Forward slice against selection represents *sufficiency*
- Slicing is inherently tied to an execution of a program
 - Forward slicing produces an execution trace
 - Backwards slicing consumes an execution trace

$$\rho, e \nearrow_T V$$

Forward Eval

$$v \searrow_T \rho, e$$

Backward Eval

Background: Galois Connections

- Forwards (fwd) and backwards (bwd) slicing form a Galois connection
 - Pair of functions between partially ordered sets
 - Partial orders are on the selection states of values
- Admit the notion of De Morgan Dual: $f^\circ = \neg_B \cdot f \cdot \neg_A$
- Nice round-tripping properties:

$\circ fwd \circ bwd(V) \geq V$

$\circ bwd \circ fwd(V) \leq V$

V represents selected parts of a value

-2	-1	0
-1	1	1
0	1	2

 $\leq$

-2	-1	0
-1	1	1
0	1	2

output

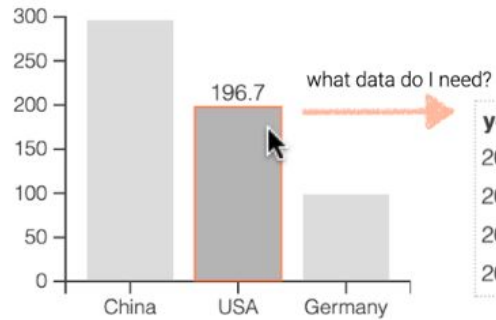
5	4	2	5	2
3	1	2	-1	-2
3	0	1	0	-1
2	1	-2	0	0
1	0	-1	-1	-2

output

5	4	2	5	2
3	1	2	-1	-2
3	0	1	0	-1
2	1	-2	0	0
1	0	-1	-1	-2

Linked Outputs via De Morgan Dual

View 1

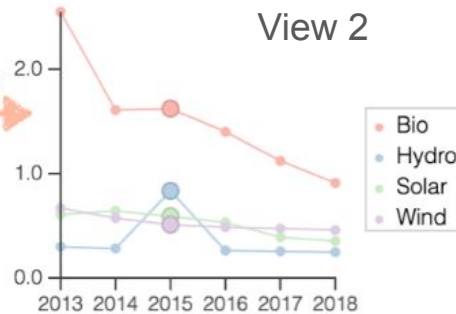


View 0

year	country	energyType	output
2015	USA	Bio	16.7
2015	USA	Hydro	80
2015	USA	Solar	26
2015	USA	Wind	74

what needs this data?

View 2



- Intuitively:

- Backward slice against selected output in View 1
- Negate the result of backward slicing in View 0
- Forward slice against the negated selection into View 2
- Negate the result of forward slicing in View 2

The Big 4

- $bwd(V)$ = “what do I need?”
- $bwd(V)^\circ$ = “what do **only** I need?”
- $fwd(V)$ = “what am I sufficient for?”
- $fwd(V)^\circ$ = “what am I necessary for?”

Performance And Complexity

- Repeated slicing costly, as need to rewind and replay the entire computation every time
- Counter intuitive to replay entire program every time when only interested in some small part
- Slicing is tricky to formulate⁽²⁾

$$\begin{array}{c}
 \Downarrow\text{-apply} \\
 \frac{
 \begin{array}{c}
 v \Downarrow_{T'} \rho_1 \cdot \rho_2 \cdot \rho_3, e, \beta \quad \rho_3, e, \beta \Downarrow_w v', \sigma \quad v' \Downarrow_U \rho, e_2, \alpha \\
 \rho_2 \Downarrow \rho'_1, h, \beta' \quad \text{cl}(\rho_1 \sqcup \rho'_1, h, \beta \sqcup \beta', \sigma) \Downarrow_T \rho', e_1, \alpha'
 \end{array}
 }{
 v \Downarrow_{T U \blacktriangleright w: T'} \rho \sqcup \rho', e_1 e_2, \alpha \sqcup \alpha'
 }
 \end{array}$$

Contents

1. Background and Problem Statement
2. Work to date
3. Work in Progress
4. Conclusion and Future Work

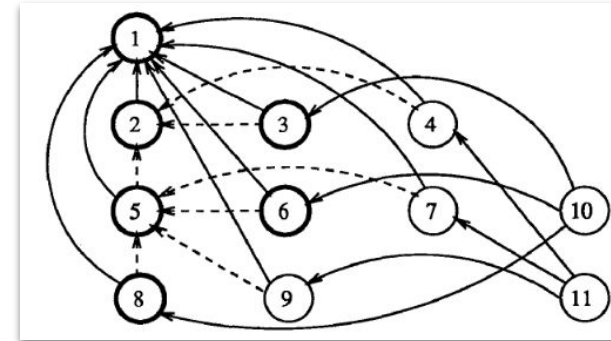
Towards Graphical Galois Slicing

Basic approach: decompose the approach into 2 steps:

1. Evaluate program to **dependence graph**
2. Perform slicing queries against nodes in the graph

Dynamic Dependence Graphs

- Formulation introduced in 1990⁽²⁾
- Primarily used for imperative programs, but not always⁽³⁾
- Nodes represent statements or expressions
- Data and control-flow dependencies as edges
- Backward slice of a set of nodes is equivalent to finding the set of reachable nodes



(2) Hiralal Agrawal and Joseph R. Horgan. 1990. Dynamic program slicing

(3) Umut A. Acar, Guy E. Blelloch, and Robert Harper. 2006. Adaptive functional programming. TOPLAS 28, 6

Galois Connections on a Graph

- Associate values with nodes in the graph
 - ◆ Edges between values A and B if A was consumed to produce B
- Define Boolean lattices on nodes associated with inputs and outputs
 - ◆ Lattices are powersets of nodes associated with inputs and outputs
- Then we have a single Galois connection between the inputs and outputs
- Define $bwd_G(V)$, $fwd_G(V)$ for slicing over the graph G of a computation

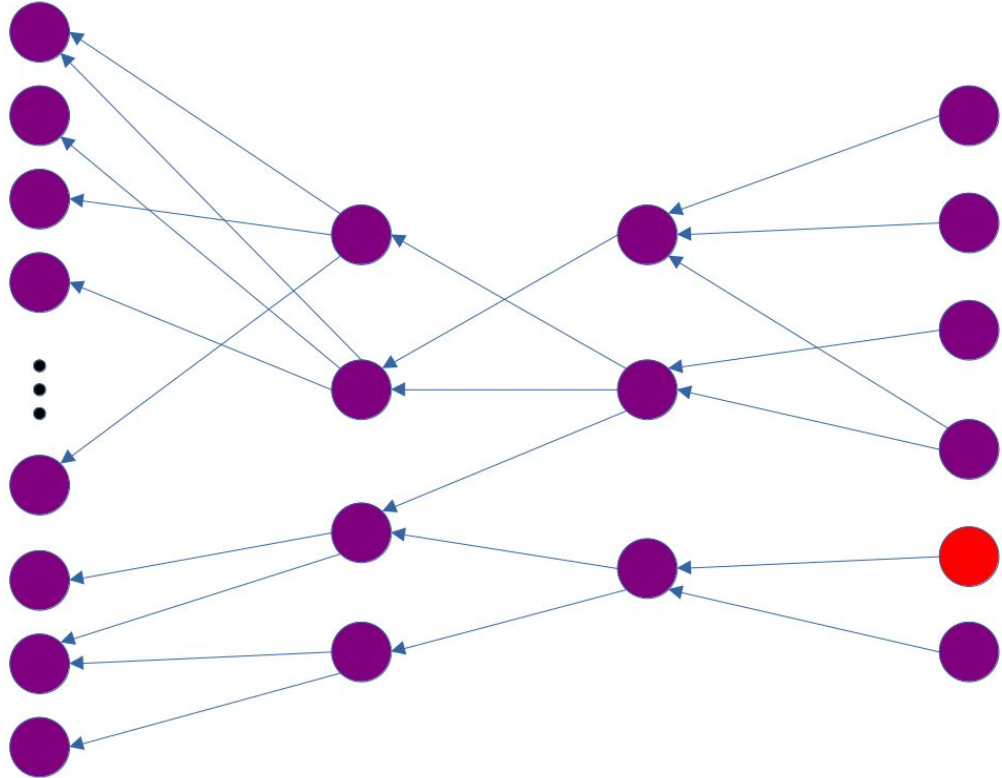
Examples

- Let's look at some examples of how slicing over the graph works
- Going to use an abstract graph instead of concrete example
- Aim is to illustrate how simple slicing algorithms are
- As a bonus at the end we demonstrate how this improves performance

Example: Backwards Slicing as Reachability

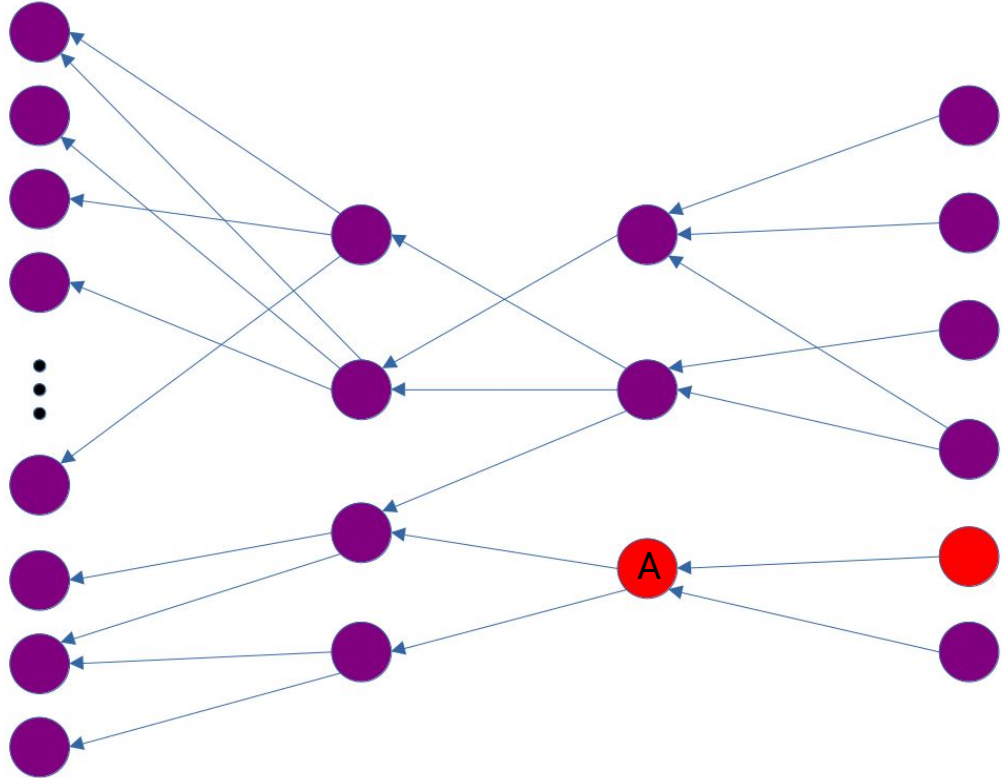
Abstract dependence graph
of a hypothetical program

We're going to slice against
the red node



Example: Backwards Slicing as Reachability

We can reach A, so we add it

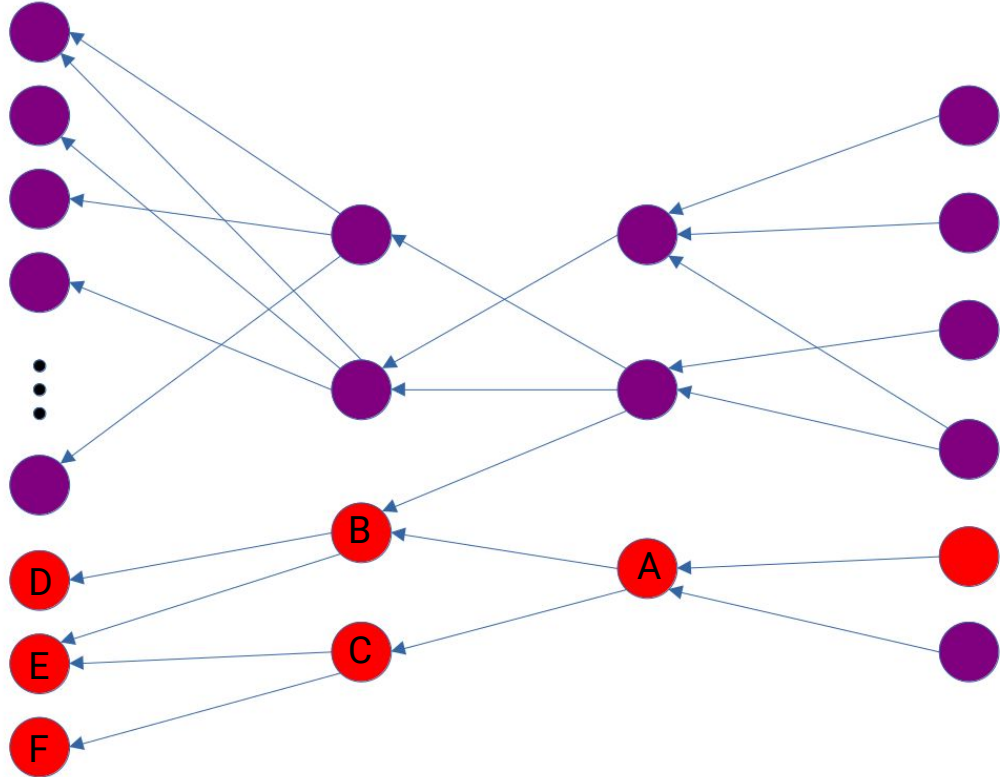


Example: Backwards Slicing as Reachability

We can reach A, so we add it

Now we can reach B and C

And finally we can reach D, E and F, the inputs relevant to our initial selection



Backwards Slicing Algorithm

$$V \triangleleft_{\text{alg}_{\hat{G}}} V'$$

where $V \subseteq S(\hat{G})$ and $V' \subseteq T(\hat{G})$

$$\frac{\triangleleft_{\text{alg}} \quad (\emptyset, \emptyset), V \triangleleft_{\text{vertices}_{\hat{G}}} G}{V \triangleleft_{\text{alg}_{\hat{G}}} T(G)}$$

$$G, V \triangleleft_{\text{vertices}_{\hat{G}}} G'$$

where $G \subseteq G' \subseteq \hat{G}$

$$\frac{\triangleleft_{\text{vertices-done}}}{G, \emptyset \triangleleft_{\text{vertices}_{\hat{G}}} G}$$

$$\frac{\triangleleft_{\text{vertices-skip}} \quad G, V \triangleleft_{\text{vertices}_{\hat{G}}} G'}{G, \alpha \cdot V \triangleleft_{\text{vertices}_{\hat{G}}} G'} \quad \alpha \in V(G)$$

$$\frac{\triangleleft_{\text{vertices-extend}} \quad G \uplus H, V' \uplus V \triangleleft_{\text{vertices}_{\hat{G} \uplus H}} G'}{G, \alpha \cdot V \triangleleft_{\text{vertices}_{\hat{G} \uplus H}} G'} \quad H = \{\alpha : V'\}$$

Backward Slicing Performance

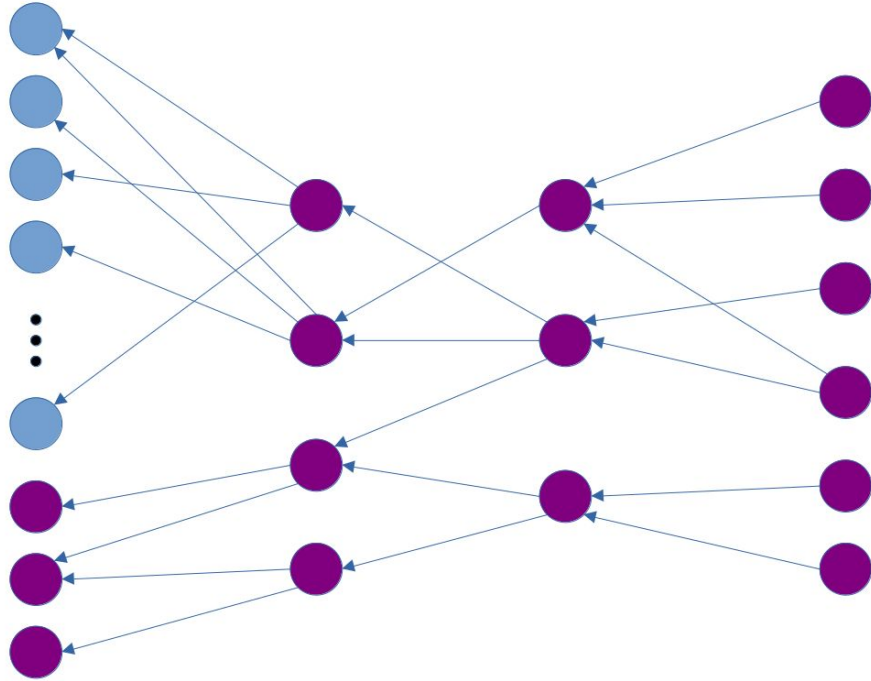
Test-Name	T-Eval	T-Bwd	G-Eval	G-Bwd
slicing/convolution/edgeDetect	741.00	503.50	1290.20	5.00
slicing/convolution/emboss	626.80	409.60	1084.50	4.50
slicing/convolution/gaussian	647.90	415.10	1098.40	4.00
graphics/grouped-bar-chart	83.70	64.10	134.10	1.50
graphics/line-chart	142.30	96.50	232.20	2.20
graphics/stacked-bar-chart	47.00	38.00	97.80	1.50
slicing/dtw/compute-dtw	163.10	133.90	317.70	19.60

Naive Forward Slicing

- De Morgan dual allows for naive implementation of forwards slicing:
 - In graph setting $fwd_G = bwd_{G^{op}}^\circ$
- No need to define a separate forward slicing procedure
 - The two parts of a Galois connection uniquely determine each other mathematically
 - This does not necessarily suggest an efficient implementation

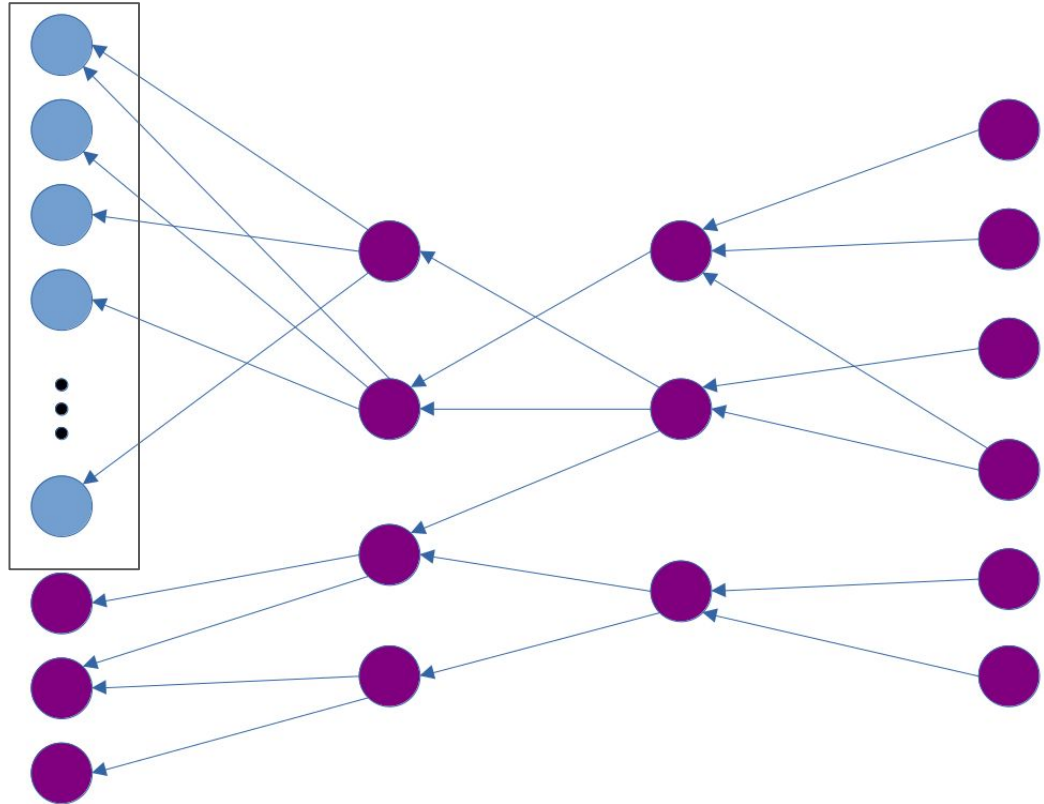
Naive Forward Slicing

Want to forward
slice the blue nodes



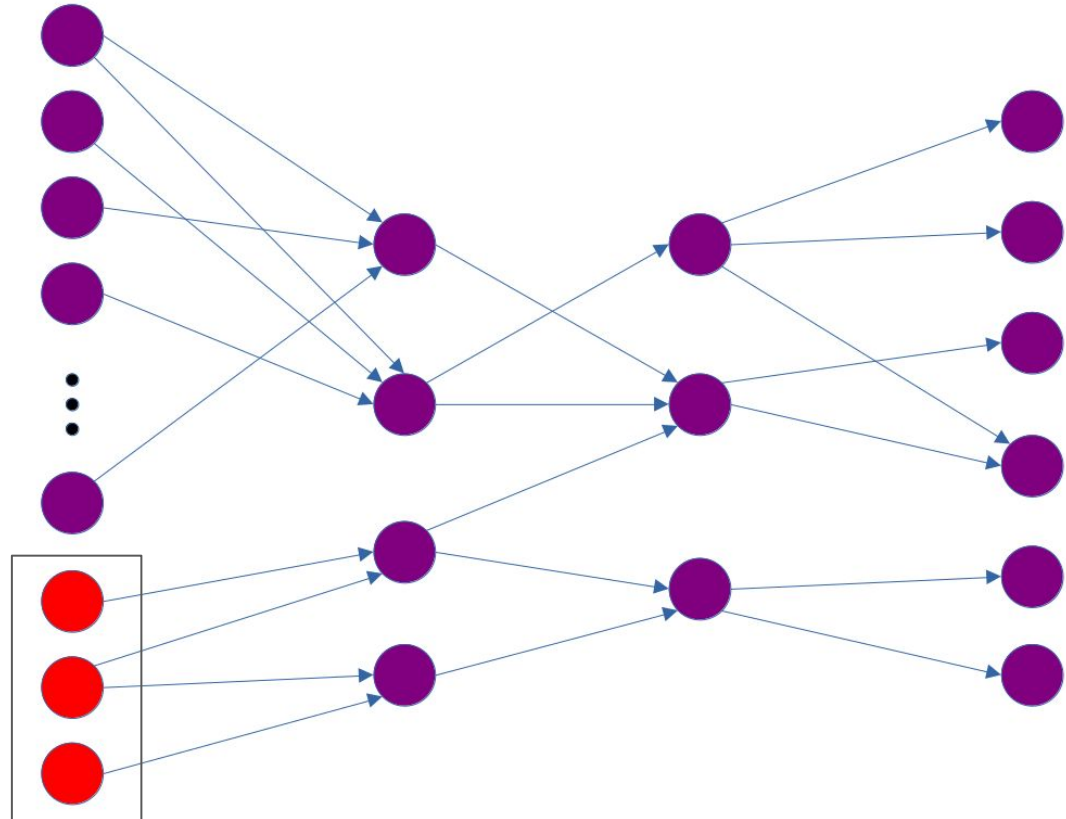
Naive Forward Slicing

1. Select relevant nodes



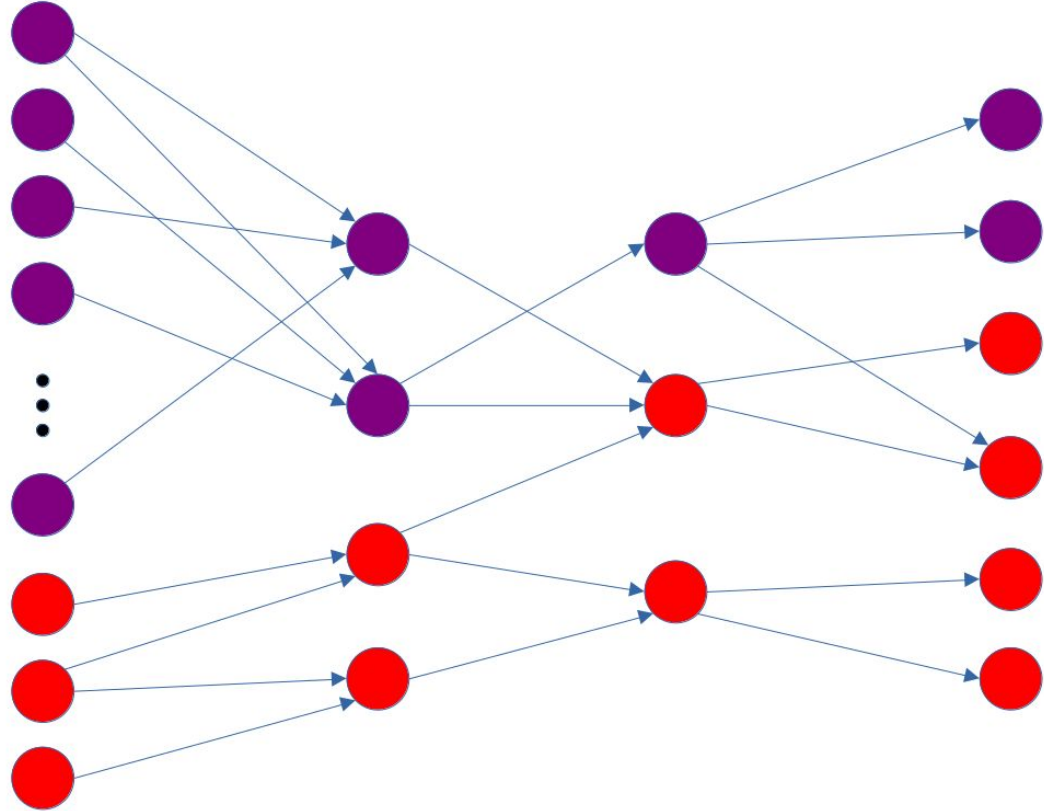
Naive Forward Slicing

1. Select relevant nodes
2. Flip selection and edge directions



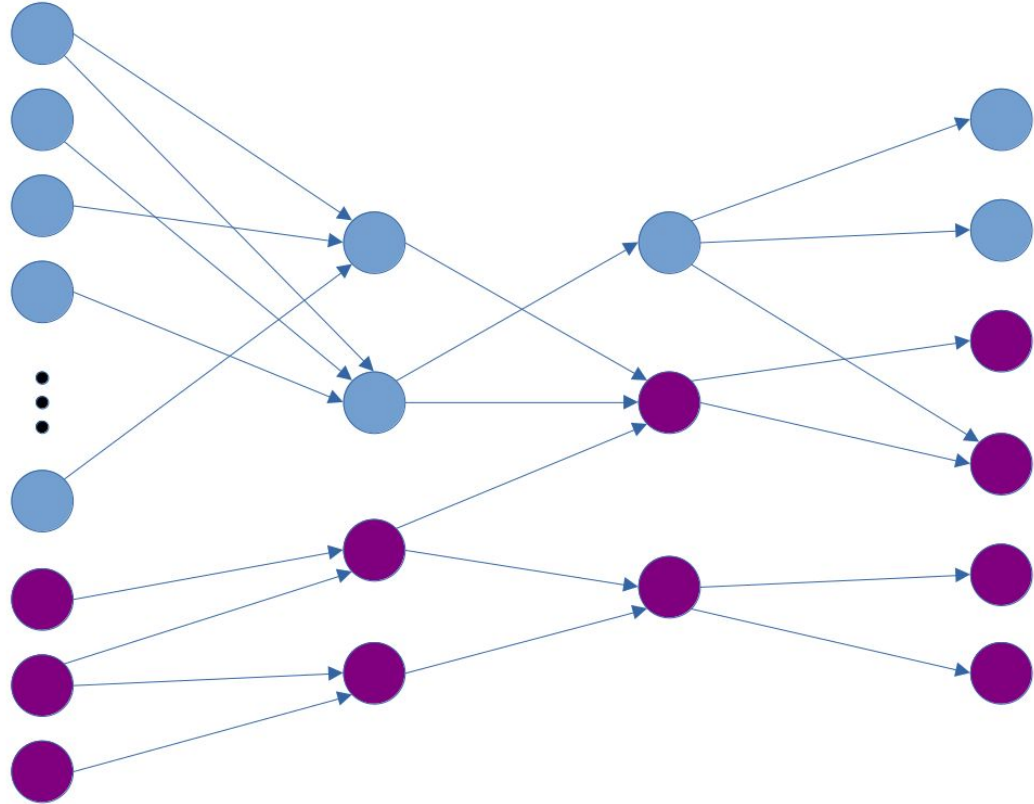
Naive Forward Slicing

1. Select relevant nodes
2. Flip selection and edge directions
3. Run backward slicing!



Naive Forward Slicing

1. Select relevant nodes
2. Flip selection
3. Invert edges
4. Run backward slicing
5. Re-invert selection



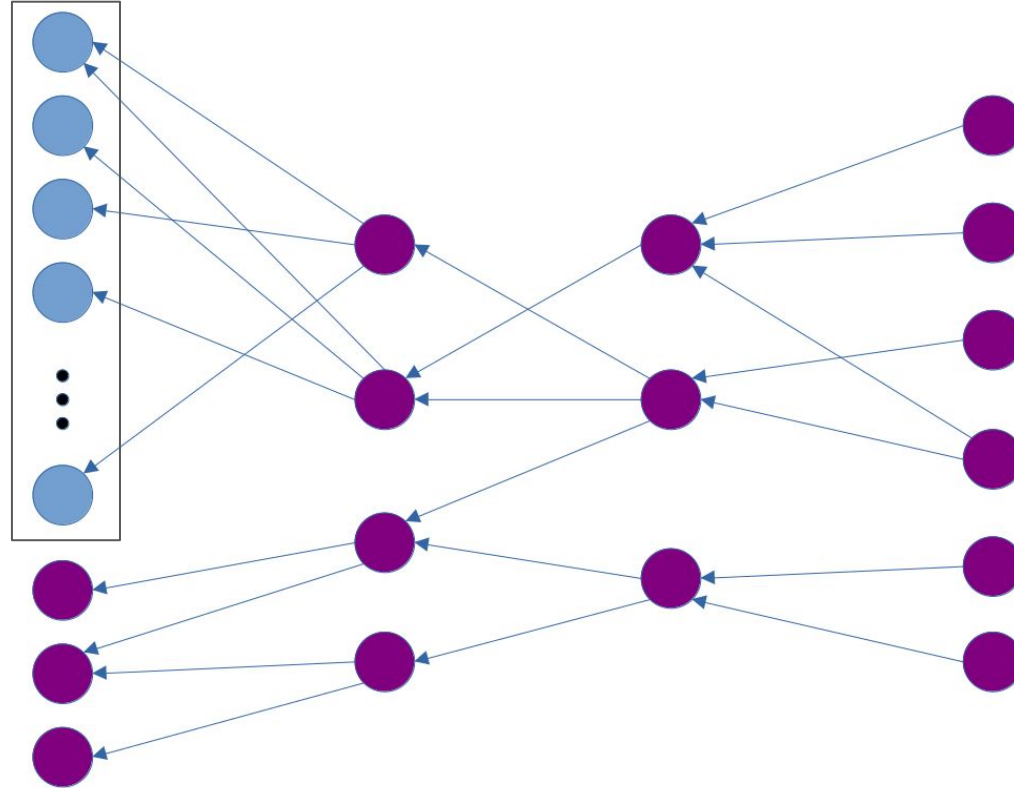
More Efficient Forward Slicing

- The prior implementation can have performance trouble
- Instead, we define a more efficient direct procedure

Test-Name	T-Eval	T-Fwd	G-Eval	G-Fwd	Naive-Fwd
slicing/convolution/edgeDetect	741.00	638.40	1290.20	3.70	2097.40
slicing/convolution/emboss	626.80	567.40	1084.50	1.40	1945.70
slicing/convolution/gaussian	647.90	569.10	1098.40	1.80	1934.50
graphics/grouped-bar-chart	83.70	79.30	134.10	147.30	1862.40
graphics/line-chart	142.30	111.10	232.20	672.50	3157.40
graphics/stacked-bar-chart	47.00	46.10	97.80	98.10	126.70
slicing/dtw/compute-dtw	163.10	194.10	317.70	17.70	176.90

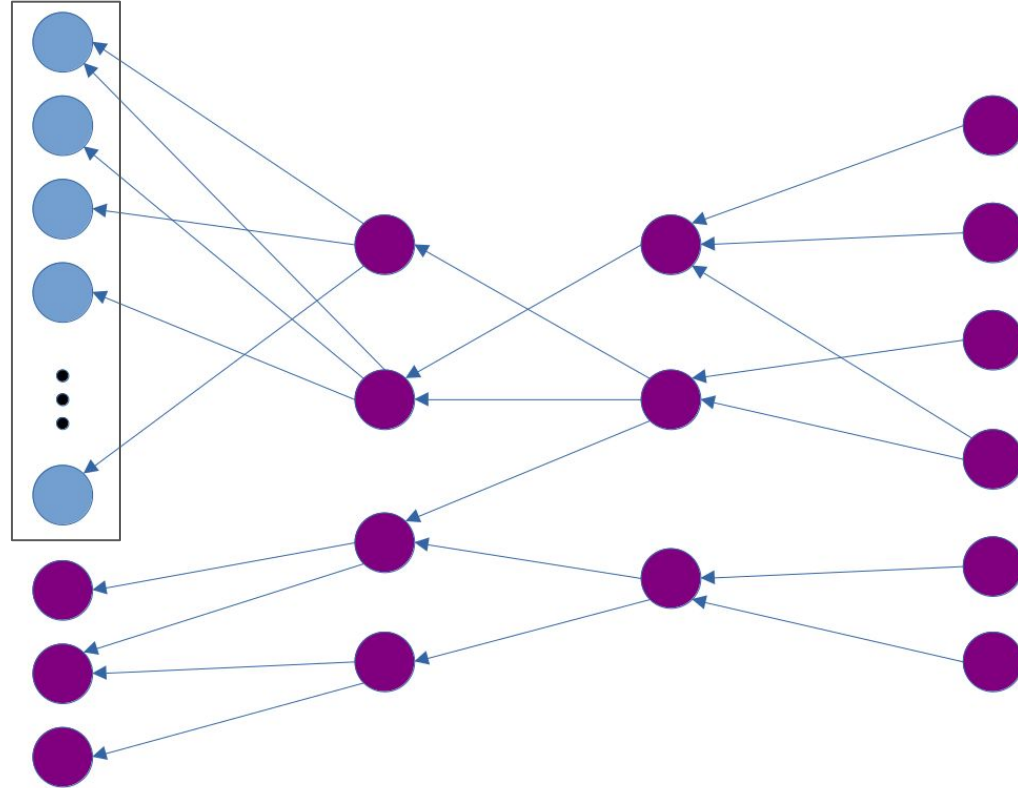
Efficient Forward Slicing

Slicing with respect to blue nodes again, but this time with the more efficient implementation



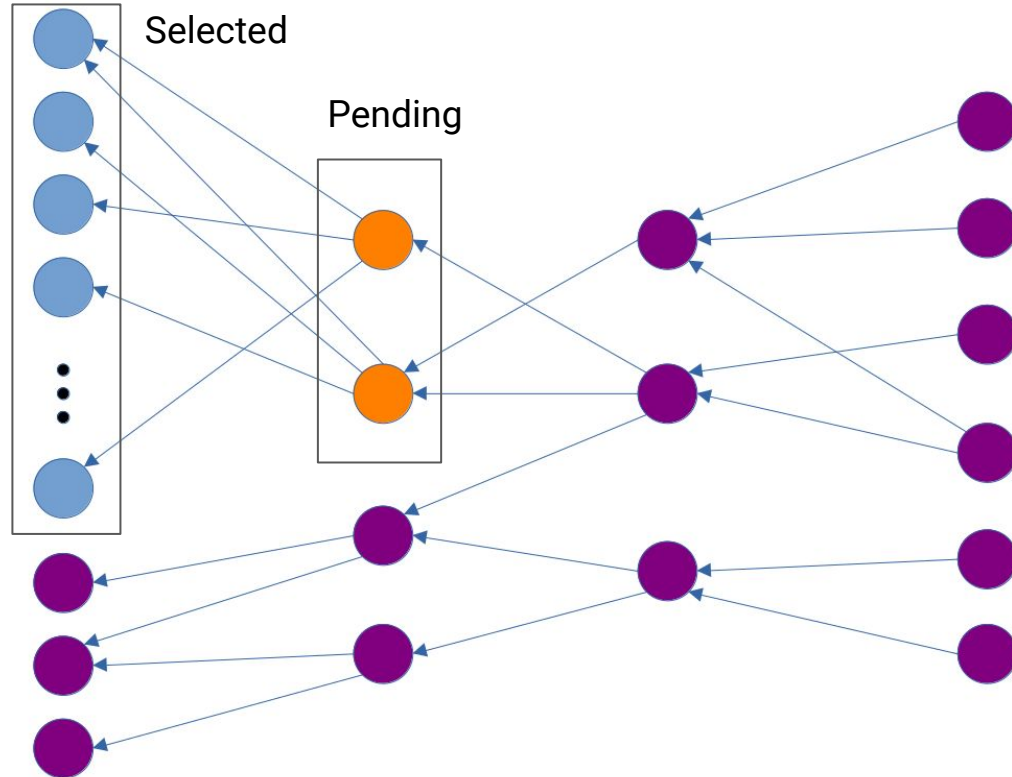
Efficient Forward Slicing

1. Enqueue neighbors of blue set



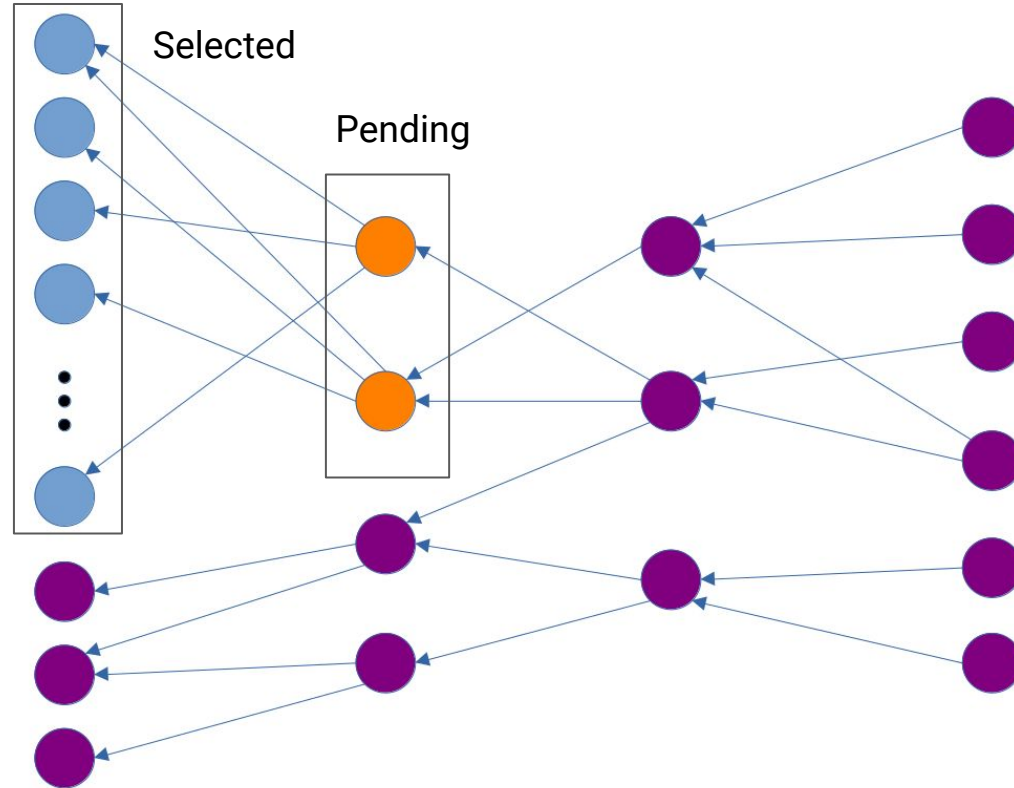
Efficient Forward Slicing

1. Enqueue neighbors of blue set



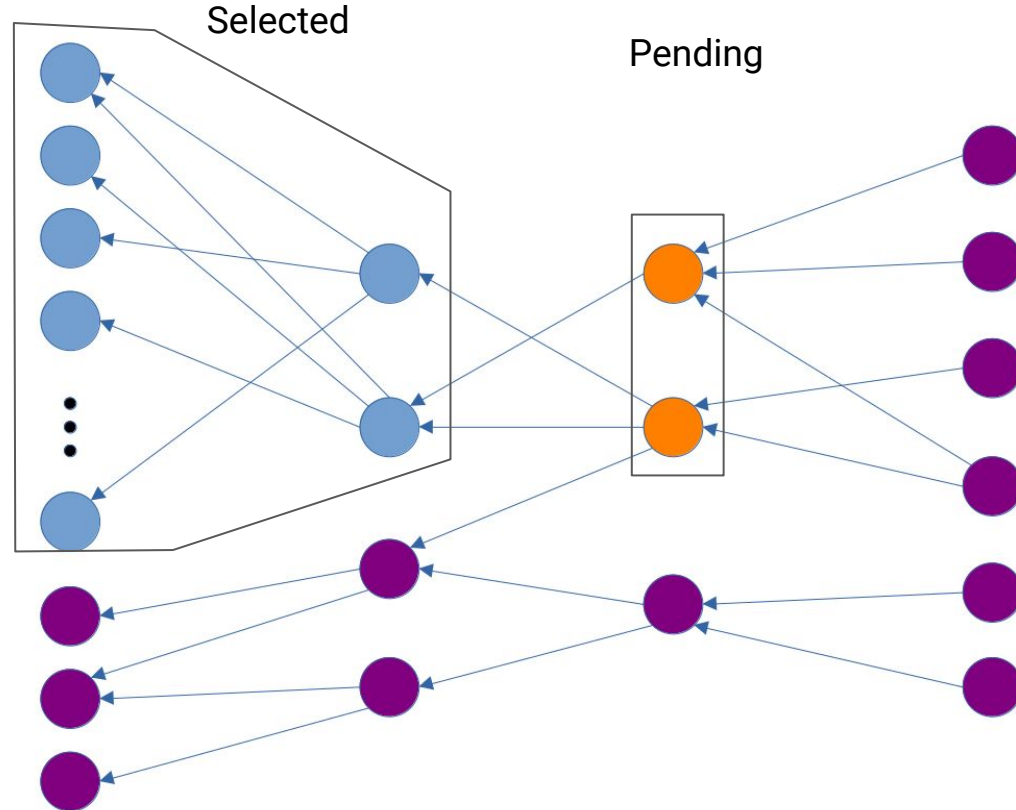
Efficient Forward Slicing

1. Enqueue neighbors of blue set
2. Ancestors of orange are subset of blue, add orange to blue



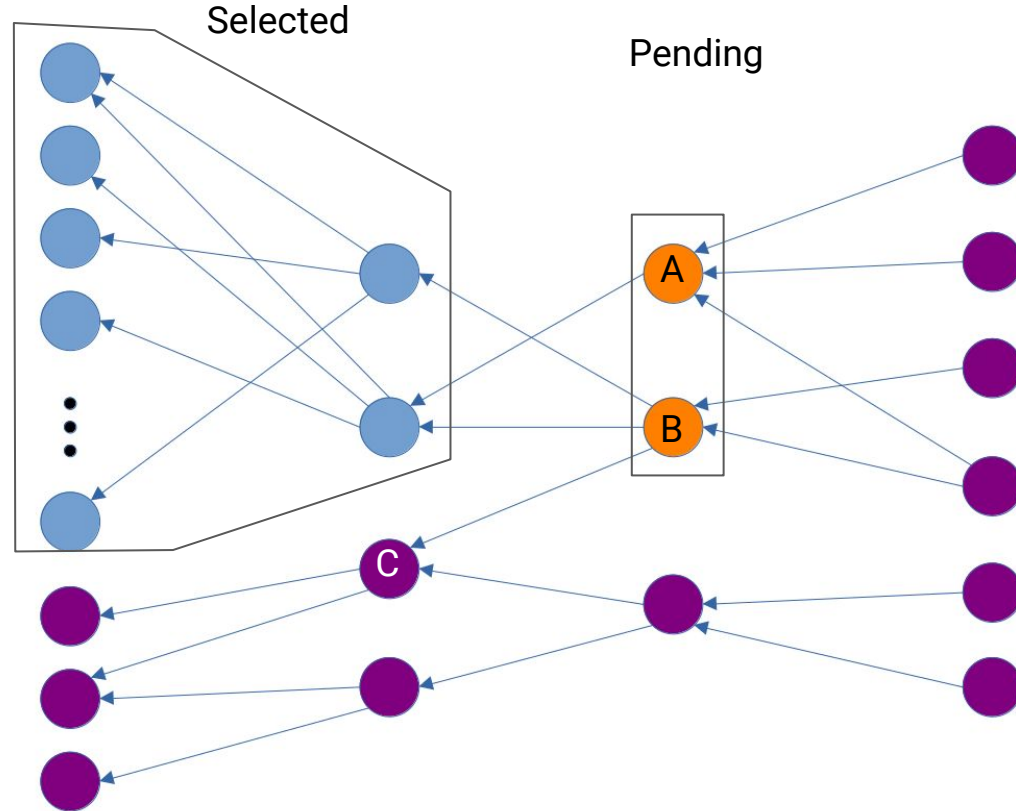
Efficient Forward Slicing

1. We enqueue the neighbors of the blue set
2. Ancestors of orange are subset of blue, add orange to blue
 - a. Colour their neighbors orange



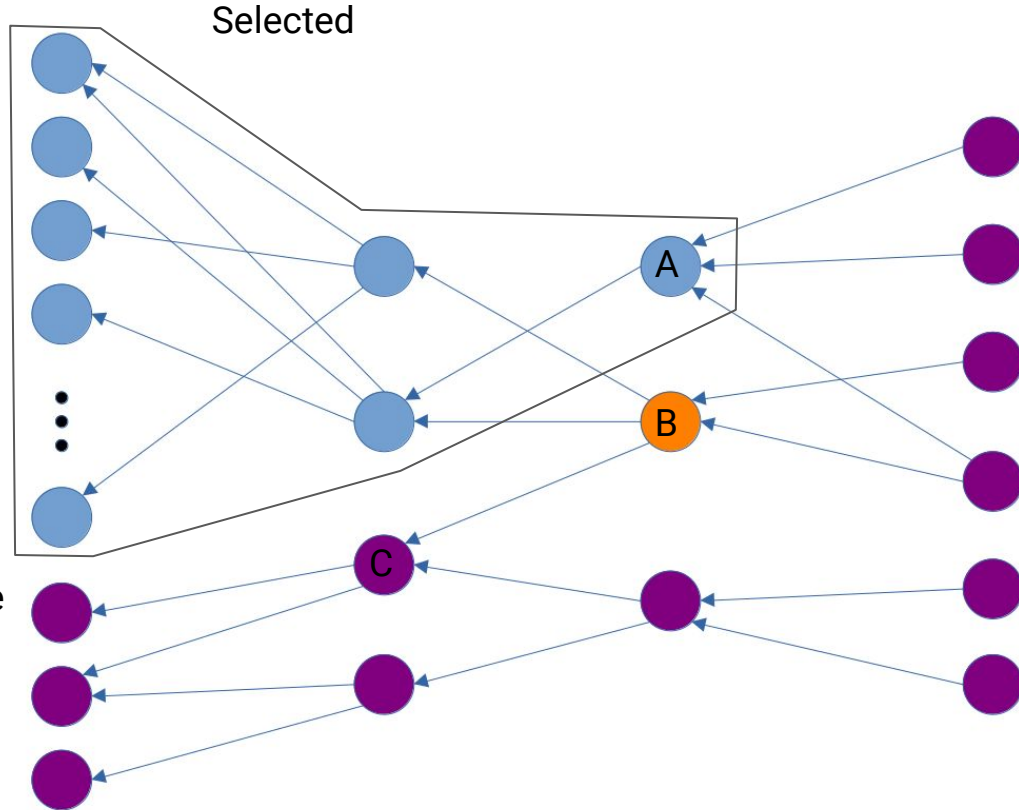
Efficient Forward Slicing

1. Enqueue neighbors of blue set
2. Pending nodes have all neighbors selected, so select them add their neighbors to pending
3. We have sufficient nodes to include A in selection, but insufficient for B (missing C)



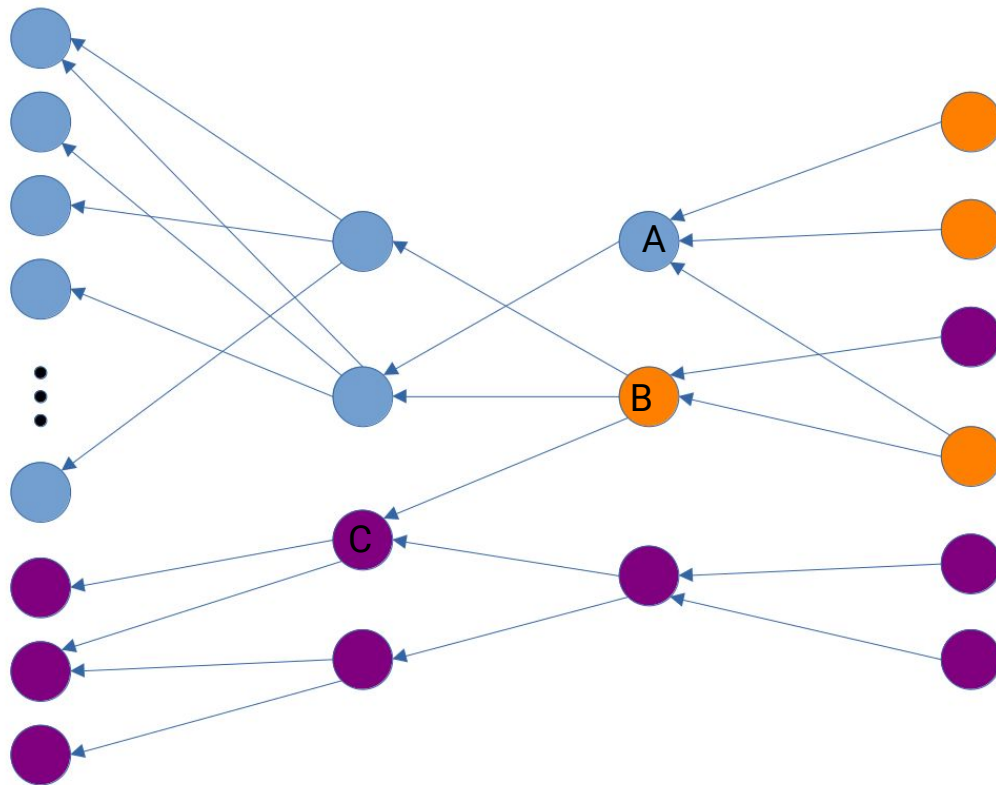
Efficient Forward Slicing

1. Enqueue neighbors of blue set
2. Pending nodes have all neighbors selected, so select them add their neighbors to pending
3. A has ancestors in blue, colour A blue, B is missing C, keep B orange
4. Repeat until done



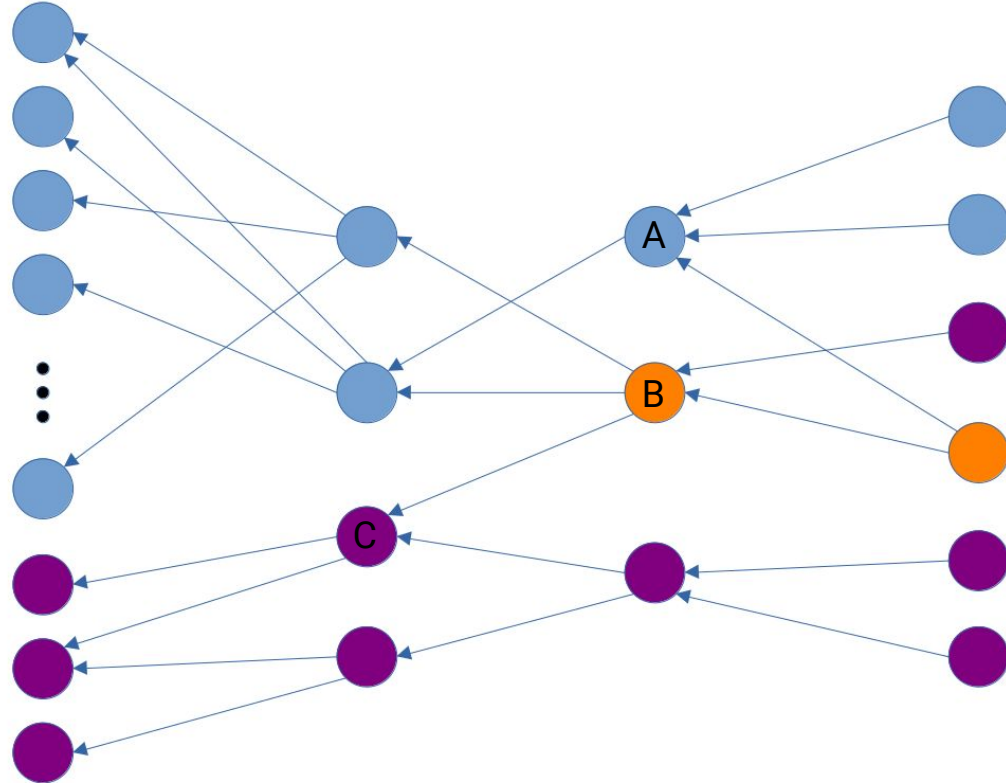
Efficient Forward Slicing

1. Enqueue neighbors of blue set
2. Pending nodes have all neighbors selected, so select them add their neighbors to pending
3. A has ancestors in blue, colour A blue, B is missing C, keep B orange
4. Repeat until done



Efficient Forward Slicing

1. Enqueue neighbors of blue set
2. Pending nodes have all neighbors selected, so select them add their neighbors to pending
3. A has ancestors in blue, colour A blue, B is missing C, keep B orange
4. Repeat until done



Recasting Linked Outputs on the Graph

- In POPL22, composed backward slicing with the De Morgan dual of forward
- In graph setting, $fwd_G = bwd_{G^{op}}^\circ$
- $fwd_G^\circ = bwd_{G^{op}}^{\circ\circ} = bwd_{G^{op}}$ since $(-)^{\circ}$ is its own inverse
- Lets us compute linked outputs for computations G, H just using backward:
 - $bwd_{H^{op}}(bwd_G(V))$

Linked Inputs

- If we can link outputs together, can we link inputs too?
- Given an input selection, what other inputs are necessary for the outputs I am necessary for?
- Implemented with our operations as: $fwd_G^\circ(bwd_G(V)) = bwd_{G^{op}}(bwd_G(V))$
- As in linked outputs, we need the dual of forward

Linked Inputs Example Code

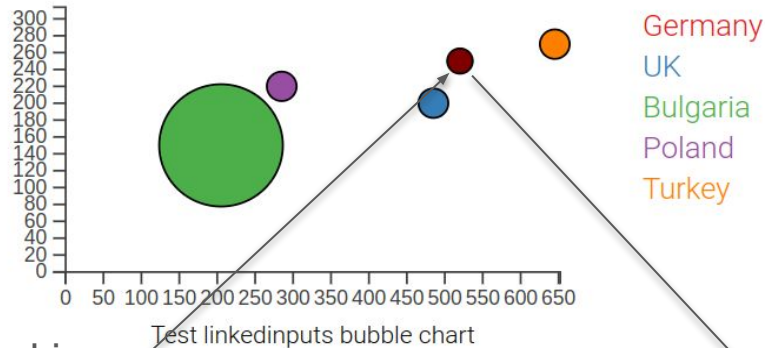
```

1 let country_water c_name =
2   let city_water = sum [city.water | country <- countries
3     , c_name == country.name
4     , city <- cities
5     , city.country == country.name];
6   farm_water = sum [country.farms | country <- countries, country.name == c_name ]
7   in head [{name: c_name, water: city_water + country.farms, farms: country.farms, popMil: country.popMil }
8     | country <- countries, country.name == c_name ];
9
10 let waters =
11   let c_names = [country.name | country <- countries]
12   in map country_water c_names
13
14 in BubbleChart {
15   caption: "Test linkedinputs bubble chart",
16   data: [{x: country.water, y: country.farms, z: country.water `div` country.popMil, c: country.name} | country <- waters]
17 }

```

Each bubble in the chart has 4 fields, drawing data from countries and the cities within them

Linked Inputs Ex. 1



Data from this record is demanded by the red bubble

countries			
#	name	farms	popMil
1	Germany	250	81

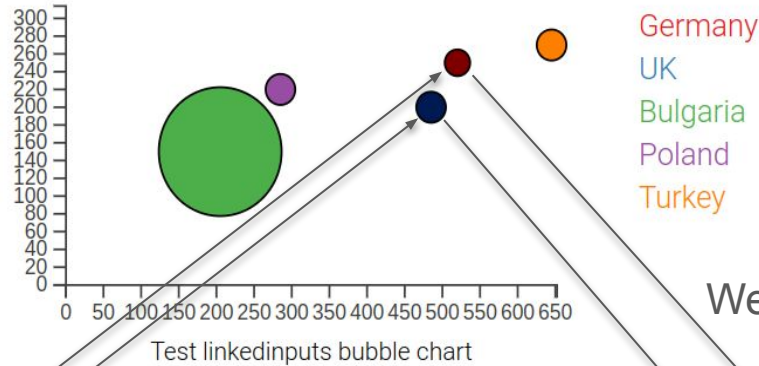
Which also demands data from the table of cities

cities			
#	name	water	country
1	Berlin	130	Germany
2	Munich	80	Germany
3	Hamburg	60	Germany

Linked Inputs Ex. 2

By growing the selection on the left

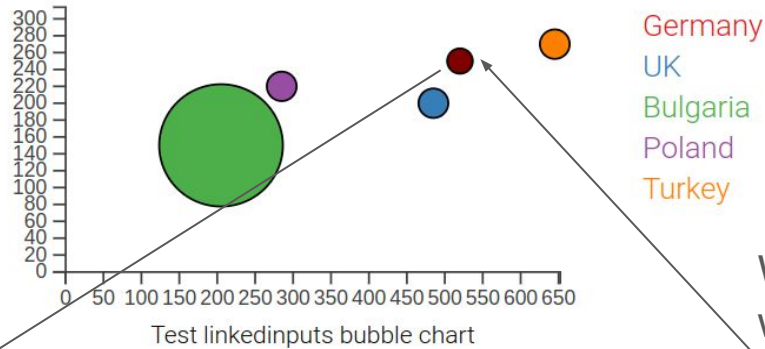
countries			
#	name	farms	popMil
1	Germany	250	81
2	UK	200	67



We pull in more linked-input

cities		
#	name	water
1	Berlin	130
2	Munich	80
3	Hamburg	60
4	London	200
5	Birmingham	50
6	Manchester	35

Linked Inputs Ex. 3



We can also go the other way

countries			
#	name	farms	popMil
1	Germany	250	81

cities			
#	name	water	country
1	Berlin	130	Germany

Contents

1. Background and Problem Statement
2. Work to date
3. Work in Progress
4. Conclusion and Future Work

Current Work: Data Linking

- Want to visualize linking an input or output back to other parts of itself
 - Simply a limitation of current implementation
- Combining linked inputs and outputs
 - Different parts of the output can and will relate different parts of the input
 - Very reasonable to combine the two approaches into one

Contents

1. Background and Problem Statement
2. Work to date
3. Work in Progress
4. Conclusion and Future Work

Future Work: Intensional Information & Graph Semantics

- Currently slicing queries only operate on extension of the graph
 - Graph structure is not motivated semantically, we reimplement the POPL22 dependency relation
- Would like to apply meaning to internal nodes
 - Does an internal node represent a meaningful subexpression of the original program?
- Fixing this involves replacing/modifying notion of dependency
 - Ideally, won't affect slicing algorithms much

Conclusion

- Dynamic dependence graphs improve performance over trace based slicing
 - Also simplify slicing algorithms significantly
- Can still exploit the properties of Galois connections on the graph
- Can use these to implement efficient linking of visualizations with only a single procedure